ReliQuary: Context-Aware Cryptographic Memory and Trust-Gated Multi-Agent Access Control

SWAYAM SINGAL[1], DR M THENMOZHI[2]

[1]School of Computer Science, VIT Vellore, Vellore, Tamil Nadu, India
[2]School of Social Sciences and Languages, VIT Vellore, Vellore, Tamil Nadu, India
Corresponding author: Dr M Thenmozhi, thenmozhi.nisha@vit.ac.in

Abstract

Modern secret-management systems face two distinct pressures: public-key mechanisms must migrate toward post-quantum standards, while authorization must move beyond static identity and role checks toward continuous evaluation of user, device, location, and behavioral context. This study presents ReliQuary, an open-source cryptographic-memory architecture designed to bind disclosure capability to an inspectable authorization process. The architecture combines encrypted vault storage, context-derived trust, multi-agent authorization, threshold key reconstruction, privacy-preserving context hooks, tamper-evident auditing, controlled sharing, and a GPU-rendered decision interface. The implementation uses FastAPI, AES-256-GCM secret envelopes, local-folder/PostgreSQL/S3-compatible adapters, sensitivity-dependent `allow`, `redact`, and `deny` decisions, per-secret passwords, expiring share tokens, Merkle structures, Python consensus modules, Rust/PyO3 cryptographic boundaries, browser and macOS clients, and a Vulkan/Dear ImGui visualizer. Reproducible verification on 13 July 2026 completed 59 Python tests with three service-dependent skips, built the Next.js website, compiled both Rust extension crates, and built the Vulkan visualizer. These results establish software-path viability but do not validate the original AWS-scale latency, machine-learning accuracy, Byzantine fault-injection, or production post-quantum claims. The original evaluation design and numerical claims are therefore retained as marked replication targets rather than current evidence. ReliQuaryis consequently a functional research platform for context-gated cryptographic memory; production use still requires hardened identity, external key management, real Circom artifacts, standardized PQC migration, independent security review, and replicated large-scale evaluation.

Keywords: cryptographic memory, zero trust, context-aware access control, post-quantum cryptography, zero-knowledge proofs, multi-agent systems, Byzantine fault tolerance, dynamic trust, secret vault, Vulkan

Introduction

In modern communication and data-storage systems, the security of public-key cryptography has historically depended on the computational difficulty of integer factorization and discrete logarithms. Shor's algorithm showed that a sufficiently capable quantum computer could solve both classes of problems efficiently, motivating the development and standardization of post-quantum cryptography (PQC) (Shor, 1994; National Institute of Standards and Technology, 2024a,b). At the same time, network architecture has shifted toward zero-trust models in which no request receives implicit confidence solely because it originates from a familiar network, device, or account (Rose et al., 2020). Role-Based Access Control (RBAC) remains useful but is often too rigid to represent rapidly changing context. Attribute-Based Access Control (ABAC) is more expressive, yet large attribute sets create policy-management and conflict-resolution complexity (Sandhu et al., 2000; Hu et al., 2014; Park and Sandhu, 2004).

These problems are related. Quantum-resistant encryption protects the mathematical boundary around data, but it cannot determine whether a currently authenticated request should receive the plaintext. Conversely, a sophisticated authorization decision is of limited value if the underlying key-establishment mechanism is vulnerable to future cryptanalytic capability. A future-facing security system must therefore address both cryptographic durability and adaptive disclosure.

This paper proposes ReliQuary, a cryptographic-memory system in which access to encrypted state is conditioned on contextual evidence and an auditable decision path. The word *memory* refers to durable encrypted information together with the context, policy, provenance, and access history required to disclose it safely. The system is designed to store ordinary text, files, or folders; evaluate a request against sensitivity and trust requirements; reveal, redact, or deny the requested information; and show the resulting path through browser, macOS, and Vulkan-based interfaces.

The original research concept joined four areas: NIST-directed PQC migration, online zero-knowledge context attestation, heterogeneous multi-agent consensus, and threshold key reconstruction. The present implementation adds a concrete local-first vault, storage adapters, controlled share links, event streaming, and visual inspection. The principal contributions are therefore:

- A unified architecture that treats cryptography, contextual authorization, auditability, and disclosure as one lifecycle rather than independent services.
- A privacy-oriented context-attestation design in which device posture, geofencing, and behavioral evidence can be verified without exposing all underlying attributes.
- A heterogeneous multi-agent decision model with neutral, permissive, strict, and watchdog policy personas, together with a BFT-oriented consensus and threshold-reconstruction design.
- A deterministic, currently runnable access engine that maps sensitivity and request context to `allow`, `redact`, or `deny`, producing a machine-readable audit event for every decision.
- A local-first implementation with file/folder secrets, PostgreSQL and S3-compatible adapters, expiring share tokens, and three user interfaces, including a Vulkan/Dear ImGui decision graph.
- A reproducible verification record that separates current evidence from the unreplicated enterprise-scale performance claims of the earlier manuscript.

The remainder of the paper follows the topical organization used by the companion Memory Retrieval Cue manuscript: background and literature are separated from the technical report, methodology, implementation, results, discussion, conclusion, and disclosures.

Background

Secret stores traditionally enforce access through possession of a password, token, device key, or role. Those controls answer whether a principal has presented a recognized credential; they do not necessarily answer whether the present request is reasonable. A credential may be stolen, a device may be compromised, or a legitimate account may request an unusual resource from an implausible location. Zero-trust architecture responds by evaluating subjects, assets, and resources at each access boundary rather than treating network location as a sufficient trust signal (Rose et al., 2020, 2025).

Post-quantum migration introduces a second design concern. NIST standardized ML-KEM in FIPS 203 and ML-DSA in FIPS 204 in August 2024 (National Institute of Standards and Technology, 2024a,b). The repository currently contains a Kyber KEM implementation and a Falcon signature implementation through Rust crates. Kyber is the predecessor family of standardized ML-KEM. Falcon was selected for future standardization as FN-DSA/FIPS 206, but it is not yet a finalized FIPS standard (National Institute of Standards and Technology, 2026). The implementation must therefore be described as PQC-capable research code, not as a fully NIST-compliant production boundary.

Zero-knowledge proofs (ZKPs) provide a way to prove a statement without exposing the witness used to establish it. In access control, this can permit a client to prove that a device or location satisfies policy without submitting every raw attribute. Practical use is complicated by circuit design, trusted setup requirements for some proving systems, proving latency, artifact management, and the need to bind proofs to fresh requests (Ben-Sasson et al., 2014; Rao et al., 2022).

Multi-agent systems provide another means of avoiding a monolithic policy decision. Agents can represent distinct risk attitudes and inspect the same evidence from different perspectives. However, ordinary majority voting is insufficient when agents may fail, equivocate, or become compromised. Practical Byzantine Fault Tolerance (PBFT) provides a formal basis for reaching agreement when a bounded number of participants behave arbitrarily (Castro and Liskov, 1999; Lamport et al., 1982).

Literature Review

Post-Quantum Cryptography Integration

The NIST PQC standardization process established lattice-based cryptography as a practical foundation for quantum-resistant key establishment and signatures (Lyubashevsky et al., 2010; Alagic et al., 2022; National

Institute of Standards and Technology, 2024a,b). Existing research has examined standalone performance and integration into transport protocols such as TLS (Poppelmann et al., 2020). Most integration work remains protocol-level: a quantum-resistant primitive replaces a classical primitive while the surrounding authorization model remains unchanged.

Research gap: Cryptographic migration alone does not address contextual misuse of a valid identity. There remains a need for systems in which post-quantum key capability participates in the authorization lifecycle rather than operating as an isolated replacement.

ReliQuary contribution: The target architecture distributes key capability to an authorization quorum and authenticates inter-agent decisions. The current Rust boundary provides Kyber and Falcon operations, while the paper explicitly recognizes the need to migrate to final standardized parameter sets and independently validated libraries.

Zero-Knowledge Proofs in Access Management

ZKPs have been applied to privacy-preserving payments, decentralized identity, and credential validation (Ben-Sasson et al., 2014; Reed et al., 2022). These deployments often concern stable or comparatively long-lived attributes.

Research gap: Ephemeral, high-rate attributes such as current device health, geolocation bounds, and behavioral anomalies are difficult to verify synchronously. Offline or asynchronous proofs do not naturally fit a low-latency authorization loop (Rao et al., 2022).

ReliQuary contribution: The proposed protocol creates request-bound proofs for contextual predicates and makes proof validity an input to watchdog-agent policy. The repository includes a deterministic development proof path and external Circom/SnarkJS hooks; real proving and verification artifacts remain deployment requirements.

Adaptive Trust Management

Limitations of static RBAC and ABAC have motivated dynamic trust engines that aggregate heterogeneous signals into continuous risk estimates (Sandhu et al., 2000; Hu et al., 2014; Park and Sandhu, 2004; Zanasi et al., 2021). A common approach uses machine learning to model nonlinear interaction among device, location, time, and behavior.

Research gap: Centralized trust engines create a single decision point and can conceal conflicts among signals. Models also drift, inherit bias from their labels, and can be manipulated by gradual behavioral poisoning.

ReliQuary contribution: The implemented trust scorer exposes weighted factors and explanations, while the proposed extension uses a calibrated classifier. Final disclosure remains subject to a separate sensitivity-aware policy and can be mediated by multiple agents.

Multi-Agent Systems for Security Enforcement

Multi-agent systems have been studied for intrusion detection, threat hunting, and distributed monitoring because they support parallelism and autonomy (Wooldridge, 2009). These applications are commonly asynchronous and reactive.

Research gap: Embedding heterogeneous agents in a synchronous, latency-sensitive authorization path requires bounded-time behavior, signed messages, consistent policy interpretation, and failure handling.

ReliQuary contribution: The research design formalizes authorization as a BFT-oriented state machine. Agents emit structured decisions and confidence values, while a quorum outcome gates disclosure and, in the target architecture, key reconstruction.

Technical Report: System Architecture

ReliQuaryis organized as five conceptual layers. This separation supports local operation today and a distributed deployment path without pretending that every target component is already production-hardened.

1. **Client layer:** browser, macOS desktop, Vulkan/Dear ImGui console, and language SDKs. Clients collect user intent and context, but do not authoritatively decide disclosure.
2. **API and identity layer:** the FastAPI entrypoint exposes health, authentication, context, trust, access, memory, sharing, agent, audit, and vault routes.
3. **Decision layer:** sensitivity thresholds, trust evaluation, multi-agent orchestration, and event generation determine whether a result is fully revealed, metadata-only, existence-only, or hidden.
4. **Cryptographic layer:** AES-256-GCM protects payload envelopes; Rust/PyO3 modules expose Kyber, Falcon, and Merkle operations; Circom/SnarkJS remains an external proving path.
5. **Storage layer:** local directories, PostgreSQL, or S3-compatible object storage persist vault and secret records. Audit events are written to JSONL and Merkle-oriented log structures.

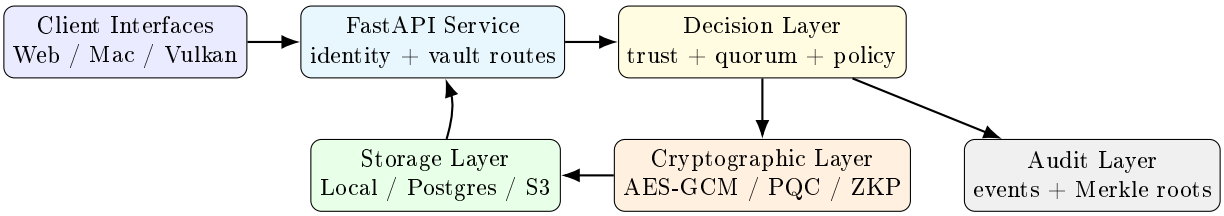

Figure 1: High-level ReliQuaryarchitecture. The API remains the security authority; interfaces display decisions but do not bypass them.

Enterprise Integration

The target design can operate as a Policy Enforcement Point downstream of an enterprise identity provider, consistent with NIST zero-trust concepts (Rose et al., 2020). LDAP, SAML, OAuth 2.0, WebAuthn, or decentralized-identifier assertions can establish identity, after which ReliQuaryevaluates request context and resource policy. A client SDK or sidecar can mediate application access. This enterprise arrangement is an architectural target; the current local setup uses development-oriented authentication paths and must not be deployed on an untrusted network without configuration changes.

Threat Model

The design considers the following adversaries:

- **Passive quantum adversary:** records ciphertext today for future decryption. The target mitigation is standardized post-quantum key establishment and signatures.
- **Active network adversary:** reads, modifies, replays, or drops messages. Mitigations include authenticated transport, fresh request binding, signed messages, and replay-resistant proof statements.
- **Malicious or compromised client:** presents valid identity credentials while attempting access beyond legitimate need. Mitigations include contextual trust, sensitivity thresholds, per-secret passwords, and fail-closed decisions.
- **Compromised agent:** submits arbitrary or conflicting decisions. A BFT quorum of size $n \geq 3f + 1$ is intended to tolerate up to f Byzantine participants (Castro and Liskov, 1999).
- **Insider or storage adversary:** reads backing storage or tampers with audit records. Payload encryption limits plaintext exposure; Merkle-rooted logging supports tamper evidence (Merkle, 1990).
- **Share-link attacker:** obtains or guesses a token. Expiry, view limits, optional share passwords, and per-secret access passwords reduce but do not eliminate risk.

Physical side channels, coercion, compromised operating-system kernels, malicious dependency updates, and denial of service are outside the protection demonstrated by the current repository. High-assurance deployments require an HSM or Secure Enclave strategy, dependency review, key rotation, backups, rate limits, and operational monitoring.

Methodology

Context and Trust Representation

The implemented trust scorer calculates five interpretable factors: context verification c, historical behavior h, inverse risk r, consistency s, and recency t. With the default configuration, the normalized score is

$$T = 0.30c + 0.25h + 0.20r + 0.15s + 0.10t, \qquad T \in [0, 1]. \tag{1}$$

The API-facing access engine represents the score on $[0, 100]$ and applies explicit penalties for non-owner requests, remote access without a trusted local session, missing device verification, and missing biometric or explicit high-trust confirmation. This deterministic path is the current decision authority and is easier to audit than an opaque classifier.

The original adaptive model is retained as a research extension. It derives a feature vector $\mathbf{f} = [f_1, f_2, f_3, f_4]$ from device consistency, temporal anomaly, geographic velocity, and behavioral anomaly. A calibrated classifier M estimates

$$T_{ML} = 100\, P_M(\text{trustworthy} \mid \mathbf{f}). \tag{2}$$

The earlier manuscript proposed a random forest and Isolation Forest trained on a rolling 30-day window. That process remains valid as an evaluation design, but the repository does not contain the claimed enterprise dataset or an accepted trained model. Any future model must use subject- or organization-held-out validation, calibration, drift monitoring, and analyst-reviewed labels.

Sensitivity-Aware Disclosure

Each resource receives one of five sensitivity levels. Table 1 shows the current policy thresholds.

Table 1: Implemented sensitivity thresholds and disclosure behavior

Sensitivity	Required score	Policy intent
Public	0	Low-risk data may be disclosed, subject to request validation.
Private	50	Owner-oriented data requires moderate contextual trust.
Sensitive	75	High trust and a verified device are expected.
Secret	90	Verified device and biometric or explicit confirmation are expected.
Sealed	101	Direct value disclosure is impossible under the current policy.

The engine returns one of three decisions: `allow` reveals the requested value; `redact` reveals only metadata or existence; and `deny` reveals no useful resource information. The exact reason list is persisted with the event.

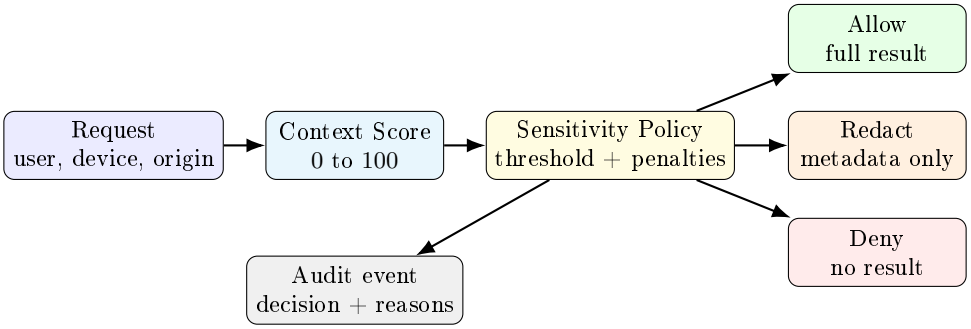

Figure 2: Implemented access-decision path used by the API and all three interfaces.

Heterogeneous Multi-Agent Consensus

The research design uses seven agents and targets tolerance of two Byzantine faults because $7 \geq 3(2)+1$. Four policy personas provide deliberately different risk attitudes:

Table 2: Agent persona policy definitions retained from the original design

Persona	Target count	Decision rule
Neutral	2	Grant if $T \geq 50$; otherwise deny.
Permissive	1	Grant if $T \geq 30$; flag $30 \leq T < 50$ for audit.
Strict	2	Grant if $T \geq 70$; otherwise deny.
Watchdog	2	Grant only if $T \geq 60$ and the context proof is valid; alert on proof failure.

The implemented test configuration also exercises smaller four-agent networks. The generic constraint $n \geq 3f + 1$ is calculated by the consensus module. The conceptual protocol is:

Listing 1: BFT-oriented multi-agent authorization protocol

```
INPUT: request r, trust T, proof pi, agents A, quorum q = 2f + 1
1. Orchestrator broadcasts (r, T, pi) to each agent.
2. Every agent applies its persona policy and returns
   (decision, confidence, authenticated response).
3. Invalid or late responses are excluded.
4. If fewer than q valid responses remain, return DENY.
5. Compare confidence-weighted grant and deny evidence.
6. Return the fail-closed consensus decision and audit record.
```

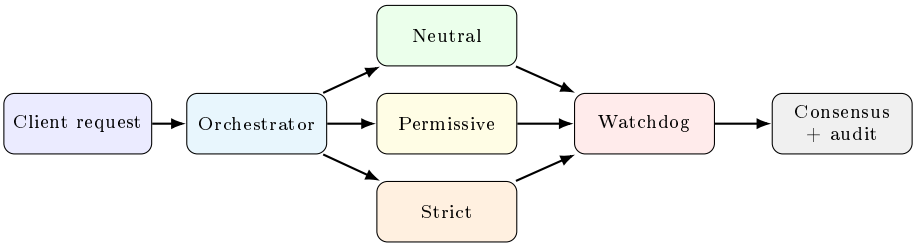

Figure 3: Heterogeneous agent evaluation and consensus sequence. In production, each response must be authenticated and bound to the request.

Consensus-Gated Threshold Cryptography

The target key-management design combines Shamir's Secret Sharing (SSS) with the agent quorum (Shamir, 1979). A vault key K is split into n shares with threshold $t = 2f + 1$. Share s_i is encapsulated for agent a_i. Reconstruction proceeds only after an approving quorum:

Listing 2: Consensus-gated key reconstruction

```
INPUT: consensus D, approving agents A+, threshold t
1. If D is not GRANT or |A+| < t, return failure.
2. Request one protected key share from each approving agent.
3. Authenticate each response and decapsulate its share.
4. Reconstruct K from any valid threshold subset.
5. Use K only for the authorized operation, then erase transient copies.
```

The repository tests SSS reconstruction and rejection with insufficient shares. Full coupling between distributed PQC-protected shares and every vault read remains a target architecture rather than a completed production key-management system.

Zero-Knowledge Context Attestation

The intended online protocol binds a proof to a request nonce, policy identifier, and short validity interval. Public inputs contain only the predicates needed by policy; private inputs contain raw contextual evidence. A verifier supplies the result to the trust engine and watchdog agents. The current repository supports deterministic development envelopes and optional Circom/SnarkJS integration. Mock batch verifiers are not cryptographic evidence and are excluded from production claims.

Secret Storage and Sharing

Vault records can contain text, a single file, or a folder. File and folder inputs are packaged for transport and encrypted into a secret envelope. A secret may additionally require its own password. Share tokens contain an expiry, maximum view count, and optional share password; opening a token may also require the underlying secret password. Cross-device sharing works only when both devices can reach the same API and backing storage. The repository is not a decentralized global relay or a replacement for end-to-end messaging protocols.

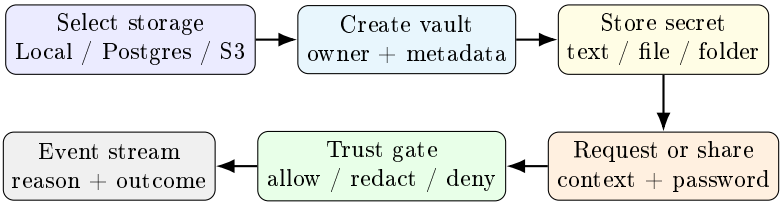

Figure 4: End-to-end vault workflow represented in the GUI and API.

Implementation

Backend and Storage

The canonical backend entrypoint is `apps.api.main:app`. It exposes routers for authentication, context, trust, access decisions, local-memory indexing, sharing, agents, audit, and vault operations. The local adapter provides a clone-and-run path; PostgreSQL demonstrates a real queryable database; S3-compatible storage supports object-store deployments when credentials and bucket policy are configured. IPFS and Arweave boundaries fail explicitly until configured instead of silently pretending to persist data.

Secret values use AES-GCM envelopes. GCM supplies authenticated encryption, so tampering with either ciphertext or authentication tag is detected (Dworkin, 2007). The production weakness is not the basic primitive but key lifecycle: development-oriented local key configuration must be replaced with an OS keychain, Secure Enclave/HSM, or managed KMS design before real high-value secrets are stored.

Rust Cryptographic Boundary

Two PyO3 crates are present. `reliquary_encryptor` exposes AES-GCM, Kyber, Falcon, hashing, and zeroization-related operations; `reliquary_merkle` exposes Merkle functionality. The Python wrapper fails loudly when optional PQC modules are unavailable rather than substituting fake post-quantum output. Both crates compile successfully in the current verification, but each presently reports zero native Rust unit tests. Python integration tests cover available round trips and tamper rejection; audited test vectors and native crate tests remain necessary.

Web, macOS, and Vulkan Interfaces

The browser console and native Tk-based macOS GUI provide storage initialization, vault creation, secret operations, trust controls, and event inspection. They are clients of the same API and do not store authoritative secret state themselves.

The Vulkan application is implemented in `visualizer/vulkan/src/main.cpp`. GLFW creates the window and surface; Vulkan initializes the instance, device, swapchain, image views, render pass, framebuffers, command buffers, and synchronization; Dear ImGui supplies the controls. On macOS the path uses MoltenVK. The graph displays storage, trust-gate, request, and result nodes while API calls create vaults, store text/files/folders, request secrets, create share tokens, simulate remote attackers, and refresh events. Vulkan is therefore the native visual-computing and interaction layer, not the cryptographic or policy engine (Khronos Group, 2026; Cornut, 2026).

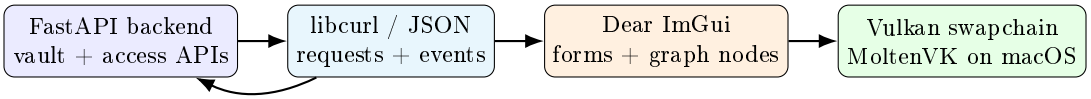

Figure 5: Role of Vulkan in ReliQuary: GPU rendering of an API-backed trust graph and unified control console.

Table 3: Current implementation status

Area	Status	Evidence and boundary
Vault API and storage	**Implemented**	Local, PostgreSQL, and S3-compatible adapters; text/file/folder payload flows.
Access decisions	**Implemented**	Deterministic sensitivity thresholds, penalties, allow/redact/deny, reasoned events.
Sharing	**Implemented**	Expiring, view-limited tokens with optional share and secret passwords.
Audit structures	**Implemented**	JSONL decision events and Merkle-oriented log/proof modules.
Multi-agent consensus	**Optional research module**	Runnable Python modules and tests; not a hardened distributed seven-node deployment.
PQC	**Optional research module**	Rust Kyber/Falcon paths; standards migration and independent validation required.
ZKP	**Optional research module**	Development proof envelopes and external Circom hooks; mock verifier paths remain.
Vulkan visualizer	**Implemented**	Native swapchain and ImGui API console build successfully on macOS.
Enterprise IAM/Kubernetes	**Proposed / not yet validated**	Architectural integration target, not current deployment evidence.

Implementation Boundaries

Evaluation and Results

Reproducible Repository Verification

The current evaluation asks a narrower and more defensible question than the original manuscript: do the implemented research paths build and behave consistently from the repository? Table 4 records the verification performed on 13 July 2026 on macOS.

The tests demonstrate important behavior rather than only importability. High-trust owner requests for secret resources are allowed; low-trust remote requests are denied; near-threshold requests can be redacted; access events are exposed to visualizers; tampered AES-GCM ciphertext and tags fail; local storage survives manager recreation; file secrets can be shared through bounded tokens; and insufficient threshold shares cannot reconstruct a secret.

These results establish a functional software research platform. They do not establish resistance to side channels, security of every dependency, correctness of unexecuted mock paths, high availability, formal BFT safety, or production-scale performance.

Security Review Findings

The repository's internal security report reaches a deliberately severe conclusion: the project is promising but not yet production-trustworthy. The strongest evidence is runnable verification and the existence of real storage and cryptographic boundaries. The principal weaknesses are prototype markers, development defaults, mock ZK paths, local key management, wildcard CORS, incomplete native tests, and uneven depth across

Table 4: Current reproducible verification results

Verification surface	Outcome	Interpretation
Python test matrix	59 passed, 3 skipped	Covers API routes, trust and access outcomes, vault persistence, sharing, AES-GCM, Merkle behavior, consensus, and threshold reconstruction. Three external-service-dependent SSS tests were skipped.
Python warnings	30	Six timezone deprecations and 24 LangGraph API deprecations; non-fatal but should be removed.
Website production build	Passed	Next.js compiled, type-checked, and generated the root and not-found static routes.
Rust encryptor crate	Built; 0 native tests	Compilation succeeds; absence of Rust unit tests limits native-boundary assurance.
Rust Merkle crate	Built; 0 native tests	Compilation succeeds; native test vectors are still required.
Vulkan visualizer	Passed	CMake configured and built Dear ImGui and the `reliquary_vulkan_visualizer` target.

the broad API surface. The earlier score graphic is intentionally excluded from this paper because a single composite number suggests a precision that the audit methodology does not support.

Original Enterprise Evaluation Design

The original manuscript proposed a standardized AWS study. Three `m6i.xlarge` nodes would host the control plane. Seven dedicated `m6i.2xlarge` instances would host the consensus agents, while five `c6i.4xlarge` load generators would use k6. Each test would run for 15 minutes after a five-minute warm-up, with geometric concurrency ramps, three independent runs, and 95% confidence intervals.

The retained legacy stack specified Kubernetes 1.28 on Bottlerocket OS, Python 3.10 with `asyncio` and `uvloop`, Open Quantum Safe `liboqs` 0.8.0, and an original PBFT implementation. It also named FoundationDB 7.3.23 as the primary datastore, although the architecture section named PostgreSQL, Redis, and S3-compatible storage. That inconsistency is corrected in the current implementation by documenting local-folder, PostgreSQL, and S3-compatible adapters as the supported storage paths; Redis and FoundationDB remain deployment ideas rather than evaluated dependencies.

The proposed baseline substitutes RSA-2048, ECDSA P-256, and a centralized decision engine for the PQC, ZKP, and BFT paths. The original trust study specified three datasets: 10,000 synthetic login samples with injected attacks, 5,000 de-identified enterprise VPN traces with analyst labels, and 2,000 adversarial patterns. This remains a useful evaluation protocol, but the datasets, infrastructure logs, raw measurements, and analysis code required to reproduce the reported values are absent from the repository.

Legacy Claims Requiring Replication

To preserve the original paper without presenting unsupported numbers as fact, Table 5 records its major empirical claims and their current evidence status. The detailed historical tables are retained in the appendix.

Discussion

What the Project Achieves

ReliQuarynow demonstrates more than a website or isolated cryptographic example. A user can initialize a real local or PostgreSQL-backed store, create a vault, store text or packaged file/folder secrets, add a secret-specific password, evaluate access under explicit context, observe an allow/redact/deny result, inspect the audit event, and create a bounded share token. The same flow is available through the API, browser interface, native macOS GUI, and Vulkan visualizer.

The visual layer is particularly valuable for research communication. Security failures are often invisible

Table 5: Validation status of numerical claims from the original manuscript

Original claim	Current status	Evidence needed
45.2 ms mean authorization latency at 1,000 clients	Unreplicated	Raw k6 output, infrastructure definition, versions, run logs, and statistical analysis.
97.0% F1 on real enterprise traces	Unreplicated	Licensed dataset, labels, split protocol, trained model, calibration, and held-out predictions.
Correct operation with two Byzantine agents	Partially exercised	Distributed seven-node fault injection with equivocation, delay, crash, and partition traces.
5.2 ms ZKP generation and 1.8 ms verification	Unreplicated	Circuit, proving system, artifact hashes, hardware, and benchmark harness.
Enterprise deployment readiness	Not supported	Threat modeling, penetration test, key management, secure defaults, CI evidence, incident procedures, and external review.

because an application returns only an HTTP status. The graph instead exposes the relationship among storage, resource sensitivity, trust evidence, decision, and visible result. A remote attacker simulation can be compared with a verified local request without changing the backend policy engine.

Scientific and Security Validity

The strongest scientific framing is that ReliQuaryis a testable architecture for context-gated cryptographic memory. It is not yet evidence that a multi-agent quorum is superior to a carefully engineered deterministic policy engine, nor that machine learning improves authorization safety. Those hypotheses require controlled baselines, ablation studies, calibrated error analysis, and adversarial testing.

Trust scores must not be interpreted as objective truth about a person. They are policy inputs derived from measurements and assumptions. False acceptance can expose a secret; false rejection can make critical data unavailable. Every future learned model therefore requires calibration, subgroup analysis where lawful and appropriate, drift monitoring, and a deterministic fail-closed fallback.

Likewise, the presence of Kyber, Falcon, ZKP, and Merkle modules does not automatically make the whole application post-quantum, zero-knowledge, or tamper-proof. Security properties apply only when algorithms, parameters, keys, proof artifacts, bindings, storage, transport, and operational procedures are all correct.

Limitations

The current study has the following limitations:

- The original enterprise benchmark and trust datasets are unavailable, so historical latency and accuracy values are unverified.
- Rust crates compile but contain no native unit-test suites; cryptographic assurance relies partly on Python integration tests.
- The ZK subsystem contains deterministic and mock paths. A production claim requires real circuits, ceremony/artifact documentation where applicable, and tamper/replay tests.
- The Python consensus implementation is not equivalent to a geographically distributed, adversarially tested PBFT deployment.
- Local key management and permissive development configuration are inappropriate for production secrets.
- Share links depend on a reachable central API and are not an end-to-end encrypted peer-to-peer messaging system.
- The current evaluation demonstrates software behavior, not formal security proof or certification.

Future Work

The immediate research agenda is ordered by evidentiary value. First, remove mock verification from production routes, add Rust test vectors, migrate PQC interfaces to finalized standardized constructions, and place keys behind macOS Keychain/Secure Enclave or an HSM/KMS abstraction. Second, deploy the seven-agent topology and publish a fault-injection harness covering crash, equivocation, delay, replay, and network partition. Third, release a lawful, de-identified trust dataset or a complete synthetic generator, use organization-held-out evaluation, calibrate probabilities, and report false acceptance and false rejection with confidence intervals. Fourth, run the preserved AWS or equivalent Apple-container-compatible benchmark protocol and publish all raw outputs. Fifth, commission external code review and penetration testing before any enterprise-readiness claim.

Longer-term work can evaluate reinforcement learning for adaptive agent thresholds and external anchoring of Merkle roots. Both ideas require caution: adaptive policies can become unstable or exploitable, while public anchoring proves only that a root existed, not that the logged event was true.

Conclusions

This paper presented ReliQuaryas a context-aware cryptographic-memory architecture that couples encrypted storage to explicit disclosure policy, interpretable trust evidence, multi-agent authorization research, threshold reconstruction, and auditable visualization. The work preserves the original post-quantum, zero-knowledge, and BFT research direction while aligning every present-tense claim with the repository.

The verified result is a functional local-first research platform: 59 Python tests pass, the web console builds, the Rust extension crates compile, and the Vulkan/Dear ImGui application builds. Users can create vaults, store text/files/folders, apply per-secret passwords, request or share secrets, and observe why disclosure was allowed, redacted, or denied. The unverified result is the earlier claim of enterprise-scale latency, 97% trust-classification F1, complete two-fault Byzantine resilience, and production readiness. Those claims are retained as explicit replication targets.

The central contribution is therefore architectural and experimental: ReliQuarysupplies a concrete environment in which cryptographic durability, contextual authorization, heterogeneous decision-making, storage, sharing, and visual explanation can be tested together. Its path to production is measurable, but not yet complete.

Additional Information

Disclosures

Human subjects: The repository verification and software study reported here did not involve human participants or tissue. Any future use of enterprise behavioral traces or biometric context must receive appropriate institutional, legal, and privacy review.

Animal subjects: This study did not involve animal subjects or tissue.

Conflicts of interest: The authors declare no financial relationship or other activity that appears to have influenced the submitted work.

Funding: No external financial support is claimed for the repository verification reported in this paper.

Data and code availability: The implementation, tests, build scripts, and documentation are maintained in the open-source ReliQuary repository. Historical enterprise datasets and AWS benchmark artifacts described by the original manuscript are not currently available and are therefore not treated as reproducible evidence.

Appendix: Historical Evaluation Record

This appendix preserves the original manuscript's numerical tables for traceability. Values in this appendix are **legacy, unreplicated claims**; they must not be cited as results of the current repository verification.

Table 6: Legacy endpoint performance claims at 1,000 concurrent users (unreplicated)

Endpoint	System	Avg ms	P95 ms	RPS	Success %
/auth/login	ReliQuary	45.2	89.7	1850	99.8
/auth/login	Classical	18.4	41.2	3210	99.9
/vaults/{id}/secrets	ReliQuary	87.6	156.2	920	99.5
/vaults/{id}/secrets	Classical	31.7	68.4	2180	99.7
/policies/{id}	ReliQuary	65.3	128.4	1100	99.6
/policies/{id}	Classical	22.1	49.8	2540	99.8

Table 7: Legacy cryptographic microbenchmark claims (unreplicated)

Operation	Avg ms	CPU %	Memory MB	Success %
Kyber-1024 KeyGen	1.2	12.5	45.2	100.0
Kyber-1024 Encaps	0.8	8.3	32.1	100.0
Falcon-1024 Sign	2.5	15.2	38.7	100.0
Falcon-1024 Verify	0.4	5.7	22.5	100.0
AES-256-GCM (1 KB)	0.1	2.1	15.3	100.0
ZKP Generate	5.2	18.6	52.3	99.9
ZKP Verify	1.8	6.9	28.4	100.0

Table 8: Legacy trust-scoring claims (unreplicated)

Dataset	Precision %	Recall %	F1 %	FPR %	FNR %
Synthetic (10K)	98.7	99.1	98.9	0.8	0.4
Real traces (5K)	97.2	96.8	97.0	1.1	0.9
Adversarial (2K)	95.8	94.3	95.0	1.5	1.2

Table 9: Legacy seven-node consensus claims with $f = 2$ (unreplicated)

Scenario	Success %	Latency ms	Rounds	Messages
Normal	100.0	45 ± 3.2	3	42
One agent faulty	100.0	68 ± 5.1	3	65
Two agents faulty	100.0	92 ± 7.8	5	89
Adversarial delay	99.8	78 ± 6.2	3–5	72
Partition (2-2-3)	99.5	105 ± 9.1	5	110

The original interpretation was that PQC operations, ZKP generation/verification, and BFT consensus imposed a 2.5–2.8× latency overhead relative to the classical baseline while maintaining interactive response times (Nielsen, 1994). It also stated that two concurrent Byzantine agents did not violate safety. Because raw data and infrastructure evidence are absent, the present paper records these statements only as hypotheses to reproduce.

Table 10: Original capability comparison, revised to show present evidence

Property	ReliQuary current	Target design	Static ABAC
Quantum-resistant crypto	Optional module	Yes	No
Dynamic trust scoring	Interpretable heuristic	Calibrated model	No
Privacy-preserving context	Development path	Real request-bound ZKP	No
BFT decision tolerance	Tested simulation	Seven-node fault-tested	No
Threshold reconstruction	Unit-tested module	Quorum-coupled keys	No
Visual decision explanation	Yes	Yes	Implementation-dependent
Production evidence	No	Required	Implementation-dependent

References

Alagic, G. et al. (2022). Status report on the third round of the NIST post-quantum cryptography standardization process. Technical Report NIST IR 8413, National Institute of Standards and Technology. https://doi.org/10.6028/NIST.IR.8413.

Ben-Sasson, E., Chiesa, A., Garman, C., Green, M., Miers, I., Tromer, E., and Virza, M. (2014). Zerocash: Decentralized anonymous payments from Bitcoin. In *2014 IEEE Symposium on Security and Privacy*, pages 459–474. https://doi.org/10.1109/SP.2014.36.

Castro, M. and Liskov, B. (1999). Practical byzantine fault tolerance. In *Proceedings of the 3rd Symposium on Operating Systems Design and Implementation*, pages 173–186.

Cornut, O. (2026). Dear ImGui. https://github.com/ocornut/imgui.

Dworkin, M. (2007). Recommendation for block cipher modes of operation: Galois/counter mode and GMAC. Technical Report NIST SP 800-38D, National Institute of Standards and Technology. https://doi.org/10.6028/NIST.SP.800-38D.

Hu, V. C., Ferraiolo, D., Kuhn, R., Schnitzer, A., Sandlin, K., Miller, R., and Scarfone, K. (2014). Guide to attribute based access control definition and considerations. Technical Report NIST SP 800-162, National Institute of Standards and Technology. https://doi.org/10.6028/NIST.SP.800-162.

Khronos Group (2026). Vulkan specification. https://registry.khronos.org/vulkan/specs/latest/html/vkspec.html.

Lamport, L., Shostak, R., and Pease, M. (1982). The byzantine generals problem. *ACM Transactions on Programming Languages and Systems*, 4(3):382–401. https://doi.org/10.1145/357172.357176.

Lyubashevsky, V., Peikert, C., and Regev, O. (2010). On ideal lattices and learning with errors over rings. In *Advances in Cryptology—EUROCRYPT 2010*, pages 1–23. https://doi.org/10.1007/978-3-642-13190-5_1.

Merkle, R. C. (1990). A certified digital signature. In *Advances in Cryptology—CRYPTO '89*, pages 218–238. Springer. https://doi.org/10.1007/0-387-34805-0_21.

National Institute of Standards and Technology (2024a). Module-lattice-based digital signature standard. Technical Report FIPS 204, National Institute of Standards and Technology. https://doi.org/10.6028/NIST.FIPS.204.

National Institute of Standards and Technology (2024b). Module-lattice-based key-encapsulation mechanism standard. Technical Report FIPS 203, National Institute of Standards and Technology. https://doi.org/10.6028/NIST.FIPS.203.

National Institute of Standards and Technology (2026). Post-quantum cryptography standardization. https://csrc.nist.gov/pqc-standardization.

Nielsen, J. (1994). *Usability engineering*. Morgan Kaufmann.

Park, J. and Sandhu, R. (2004). The UCONABC usage control model. *ACM Transactions on Information and System Security*, 7(1):128–174. https://doi.org/10.1145/984334.984339.

Poppelmann, T. et al. (2020). Post-quantum TLS without handshake signatures. In *Proceedings of the ACM Conference on Computer and Communications Security*, pages 1461–1480. https://doi.org/10.1145/3372297.3423350.

Rao, W., Zhao, L., and Zhang, Y. (2022). A survey on zero-knowledge proof systems: Applications and challenges. *IEEE Access*, 10:50807–50823. https://doi.org/10.1109/ACCESS.2022.3173394.

Reed, D., Sporny, M., Longley, D., Allen, C., Grant, R., and Sabadello, M. (2022). Decentralized identifiers (DIDs) v1.0. W3C Recommendation. https://www.w3.org/TR/did-core/.

Rose, S., Borchert, O., Kerman, A., Souppaya, M., et al. (2025). Implementing a zero trust architecture. Technical Report NIST SP 1800-35, National Institute of Standards and Technology.

Rose, S., Borchert, O., Mitchell, S., and Connelly, S. (2020). Zero trust architecture. Technical Report NIST SP 800-207, National Institute of Standards and Technology. https://doi.org/10.6028/NIST.SP.800-207.

Sandhu, R., Ferraiolo, D., and Kuhn, R. (2000). The NIST model for role-based access control: Toward a unified standard. In *Proceedings of the 5th ACM Workshop on Role-Based Access Control*, pages 47–63. https://doi.org/10.1145/344287.344301.

Shamir, A. (1979). How to share a secret. *Communications of the ACM*, 22(11):612–613. https://doi.org/10.1145/359168.359176.

Shor, P. W. (1994). Algorithms for quantum computation: Discrete logarithms and factoring. In *Proceedings of the 35th Annual Symposium on Foundations of Computer Science*, pages 124–134. https://doi.org/10.1109/SFCS.1994.365700.

Wooldridge, M. (2009). *An introduction to multiagent systems*. Wiley, 2 edition.

Zanasi, A., Sisto, R., and Cheminod, M. (2021). Machine learning for dynamic access control in industrial IoT environments. In *Proceedings of IEEE ETFA*, pages 1–8. https://doi.org/10.1109/ETFA45728.2021.9613480.